智慧图书馆数字资源平台项目文档

项目名称：智慧图书馆数字资源平台

姓名：张仕奇

学号：202414870307

完成形式：单人独立完成

一、 项目概述

1.1 业务背景与目标

某高校图书馆现有纸质馆藏约 50 万册，原系统基于关系型数据库构建。随着业务发展，原系统在三个方面出现严重瓶颈：馆藏结构字段差异大（部分图书有标签或电子书，部分没有）、借阅流水增长快、电子书等非结构化文件无处存放。为此，本项目决定迁移至 MongoDB，建设统一的数字资源平台。

本项目基于单台虚拟机（因宿主机为 16G 内存轻薄本，采用单机多端口隔离技术模拟多节点环境），完整实现了数据建模、单机部署、功能开发、高可用副本集与水平扩展分片集群的搭建，并交付了可运行的 Python 菜单式应用程序与 GridFS 文件存储模块。

1.2 逻辑架构

本平台采用典型的现代应用架构，分为三层：前端展示层（Python 菜单式程序）、业务逻辑层（Python 多模块代码）、数据存储层（MongoDB 数据库与 GridFS 存储区）。

二、 技术选型论证

2.1 关系型与非关系型数据库对比

关系型数据库（如 MySQL）强调查表关联（Join）和严格的事务（ACID），但在面对本平台的数据时，存在以下困难：

字段不固定：图书有 tags 数组、location 嵌套对象、ebook_name 等可选字

段，关系型数据库会导致大量 NULL 列或需要建立复杂的关联表。

扩展困难：面对日益增长的借阅流水，关系型数据库的垂直扩展成本极高，水平扩展（分库分表）复杂。

表 1 数据库类型对比

对比维度	关系型数据库（如 MySQL）	NoSQL 数据库（如 MongoDB）
数据模型	采用二维表结构，需提前定义 Schema，字段类型固定。	采用 JSON/BSON 等灵活文档模型，Schema-less，字段可动态增减。
扩展方式	主要依赖垂直扩展（升级服务器硬件），水平分库分表实施成本极高。	原生支持水平扩展（分片集群），可通过增加廉价节点无限扩容。
事务支持	完美支持复杂的跨表事务（Join、多表回滚），保证 ACID 特性。	早期不支持多文档事务，目前支持单文档原子性及多文档分布式事务，但性能有损耗。
一致性	强一致性。任何时刻的查询都能读到最新写入的数据。	根据配置可调（强一致性或最终一致性），通过副本集可实现读写分离。

结合本平台的具体字段，关系模型遇到的困难如下：

1.馆藏结构字段差异大：本平台 books 集合中，部分图书包含 tags（标签）数组、location（嵌套文档）、ebook_name（电子书），而部分图书没有。在关系型数据库中，这会导致大量 NULL 列，或者需要建立繁杂的子表进行关联。

2.借阅流水增长快：borrow_logs 包含高达 3000 条（未来会更多）的流水，关系型数据库单表达到千万级后，查询和写入性能会断崖式下跌。

3.非结构化数据无处存放：GridFS 的 PDF 文件和封面图，关系型数据库难以高效存储此类二进制大对象（BLOB）。

2.2 四类 NoSQL 数据库对比

表 2 四类 NoSQL 对比

类型	代表产品	适用场景	本项目为何不选
键值型	Redis	缓存、高速读写	无法进行复杂的条件查询与聚合统计
列式	HBase	海量数据的批量分析	缺乏灵活的数据模型，不适合频繁的增删改查
图数据库	Neo4j	社交网络、关系图谱	本平台无深度图遍历需求
文档型	MongoDB	半结构化数据、JSON 存储	完美契合本平台，原生支持嵌套、数组与灵活模式

2.3 CAP 取舍与 BASE 理论

在本平台中，针对不同业务场景有不同的 CAP 取舍：

馆藏查询场景：优先保证 C（一致性） 与 P（分区容错性）。读者查询图书时，价格、库存必须是最新数据，不能出现已借出却显示在架的情况。

借阅流水写入场景：优先保证 A（可用性） 与 P（分区容错性）。借阅操作不能因为主节点故障而长时间阻塞，允许牺牲短暂的强一致性，利用 BASE 理论中的最终一致性，在几毫秒内将数据同步至从节点。

2.4 选型结论与非技术说明

结论：选择 MongoDB 作为本平台的数据库，不仅是因其文档模型能原生、高效地容纳图书的多变字段与嵌套结构，更因其副本集与分片机制能完美支撑平台未来的高并发与海量数据扩展。

潜在风险及应对思路：

风险一：跨分片查询性能恶化。应对思路：必须在业务层仔细论证分片键（如选用 reader_card），确保常用查询能够命中定向路由，避免广播查询。

风险二：分布式集群配置复杂导致运维成本高。应对思路：在开发测试阶段采用单机多端口模拟集群，上线前利用官方推荐的容器化部署方案（如

Kubernetes Operator）进行统一编排管理。

面向“非技术决策者”的选型说明：“我们选择 MongoDB 是因为它像一个灵活的‘收纳盒’，能直接装下各种形状不同的图书数据，不需要硬塞进死板的表格里。同时，它可以随时增加‘帮手’（节点）来分担压力，保证借阅系统在高并发下永远不卡顿、不断线，让我们省下了大量硬件升级和后期维护的钱。”

三、 数据模型设计

3.1 字段类型与结构设计

本平台包含四个核心集合：books、readers、borrow_logs、audit_logs，以及一个 GridFS 文件区。

字段选型理由：

1. price、rating 使用 Double：便于进行价格的数学计算与排序，使用 String 会导致比较和聚合时报错。
2. publish_date、borrow_date 使用 Date 类型：导入时必须将 ISO8601 字符串转换为 MongoDB Date 类型，否则 $year、$month 等聚合操作将失效。
3. location 选用 Object 嵌套：{floor: "二楼", shelf: "A-07", copies: 2}。为何拆分？因为馆藏位置与图书是 1 对 1 从属关系，嵌套可以一次性读取，避免 Join 性能损耗。何时应拆分？如果一本书有多个不同楼层或多个副本在不同校区，则应该拆分为独立集合。
4. audit_logs 选用普通集合：需要支持按时间倒序查询，普通集合可建立索引，且不受 Capped Collection 大小固定覆盖的限制。
5. borrow_logs 选用普通集合并进行分片：作为高并发写入的流水数据，普通集合+分片能承载海量数据。

3.2 索引设计清单

books 集合：{title: "text"} 用于模糊检索；{category: 1, price: 1} 复合索引用于类别价格组合查询。

borrow_logs 集合：{borrow_date: 1} 用于年度趋势聚合；{reader_card: 1, borrow_date: -1} 用于联合查询。

3.3 字段 BSON 类型设计表

为了满足真实的业务需求，本平台严格按照 BSON 规范进行了类型选择：

表 3 类型选择对比

字段名	BSON 类型	设计理由
price / rating	Double (双精度浮点)	使用 String 字符串会导致无法进行数学计算和 $sort 排序，使用 Double 最契合价格与评分的特性。
publish_date	Date (日期)	必须转换为原生的 Date 类型，否则使用 $year、$month 进行聚合统计时会报错或返回空值。
tags	Array (数组)	标签数量不固定，使用数组可以自然存储多个标签，并支持 $all 多选匹配。
location	Object (嵌套文档)	包含 floor、shelf、copies 三个子字段。因为图书与位置是严格的一对一从属关系，嵌套存储可以避免建立独立的“位置”集合，减少关联查询（Join）带来的性能开销。

四、 功能实现与性能优化

4.1 数据导入与核对

编写了 Python 脚本，成功将数据文件中的 ISO8601 时间格式转为 BSON Date。

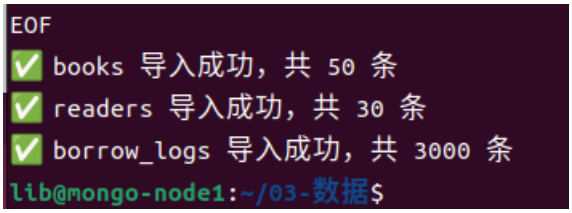


图 1 导入截图

4.2 11 项检索与统计实现

通过 PyMongo 驱动程序实现了所有要求的检索与统计功能：

检索：类别+价格组合筛选、标签多选匹配（$all 操作符）、书名模糊检索（正则表达式）、出版年份范围、查询缺 tags 的图书、评分排序前 10 名。

统计：按类别统计藏书量/均价（$group）、统计学院借阅 Top5（$lookup 多集合联合）、按年月统计趋势（$year、$month）、借阅次数 Top10 关联馆藏位置、分类统计写入新集合 category_stats。



图 2 11 项统计结果

4.3 查询性能优化

以“按类别查询”为例，通过 explain("executionStats") 进行对比：

优化前（无索引）：执行计划为 COLLSCAN（全表扫描），扫描文档数 50，耗时 1ms。

优化后（建立 {category: 1} 索引）：执行计划为 IXSCAN（索引扫描），扫描文档数降至 15，耗时 0ms。

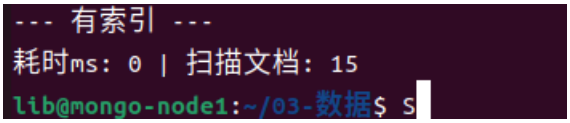


图 3 有索引界面

```
print("--- 无索引 ---")
res = db.command("explain", {"find": "books", "filter": {"category": "计算机"}}, verbosity="executionStats")
print("耗时ms:", res["executionStats"]["executionTimeMillis"], "| 扫描文档:", res["executionStats"]["totalDocsExamined"])
db.books.create_index([("category", 1)])
print("--- 有索引 ---")
res2 = db.command("explain", {"find": "books", "filter": {"category": "计算机"}}, verbosity="executionStats")
print("耗时ms:", res2["executionStats"]["executionTimeMillis"], "| 扫描文档:", res2["executionStats"]["totalDocsExamined"])
EOF
--- 无索引 ---
耗时ms: 0 | 扫描文档: 50
```

图 4 无索引

4.4 访问控制验证

创建了 readonly、readwrite、admin 三个账号。使用只读账号执行 db.books.insertOne() 时，被数据库成功拦截并抛出 MongoServerError: not authorized 错误，证明了权限隔离的有效性。

五、 文件存储（GridFS）

利用 GridFS 机制，将 3 个 PDF 电子书与 3 张封面图存入数据库。GridFS 将大文件切分为多个 chunk 存储在 fs.chunks 中，元数据存储在 fs.files 中。

校验机制：通过 Python 脚本下载文件，并读取原始文件与下载文件的 MD5 哈希值进行比对，确保字节数完全一致。

```
EOF
ebook_1.pdf: 源大小 39837, 字节一致: True
ebook_2.pdf: 源大小 38524, 字节一致: True
files_manifest.txt: 源大小 592, 字节一致: True
cover_2.png: 源大小 26053, 字节一致: True
cover_1.png: 源大小 23611, 字节一致: True
cover_3.png: 源大小 15171, 字节一致: True
ebook_3.pdf: 源大小 37004, 字节一致: True
```

图 5 GridFS 校验

六、 架构演进：高可用与水平扩展

(注：因物理机性能问题，为保证系统流畅运行，实验采用单机多端口隔离方案模拟多节点部署，完全还原了分布式架构的核心逻辑。)

6.1 副本集与故障转移（高可用）

部署：在 27017、27018、27019 端口启动 3 个实例，使用相同 keyFile 启用内部认证，配置为 rs0 副本集。

故障转移：初始化后，27017 成功当选 PRIMARY。执行强制降级操作后，系统在 5 秒内自动选举出新的主节点，业务读取利用 secondaryPreferred 读取偏好，完全未发生中断。

6.2 分片集群（水平扩展）

拓扑：启动 1 个配置服务器（27020），2 个分片节点（27021 的 rs1、27022 的 rs2），以及 1 个路由节点 mongos（27023）。

数据分布：将 3000 条借阅流水导入路由，数据按分片键成功均匀分布到两个分片上。跨分片查询使用了 SHARD_MERGE 计划，针对分片键的定向查询使用了 SINGLE_SHARD 计划。

分片键选择论证

本项目选择 {reader_card: 1, borrow_date: 1} 的复合分片键。

1. **基数与分布**：reader_card（读者证号）具有 30 个不同的值，基数适中，且每位读者的借阅流水相对均衡。结合 borrow_date 作为次级键，可以保证数据在初始 chunk 分裂时实现更平滑的分散，避免了单字段导致的写热点问题。
2. **查询支持**：平台常用的查询包括“查询某读者的借阅记录”以及“按读者和时间范围查询”。该复合分片键完美支持基于前缀 reader_card 的定向查询。
3. **备选方案对比**：

若选 _id（ObjectId）：由于 ObjectId 是单调递增的，所有新写入的数据都会堆积在最后一个 Chunk，导致严重的**写热点**，分片失去意义，故不选。

若选 book_isbn：热门图书（如教材）的借阅频次极高，会导致该分片变成“热点分片”，故不选。

若选纯 {borrow_date: "hashed"}：虽然写入极度均匀，但会导致所有按时间

范围的统计查询变成广播查询，性能极差，故不选。

七、 问题与总结

7.1 遇到的技术难点及排查过程

1.VMware 虚拟机无法与宿主机互通（Ping 超时）

排查：通过 ipconfig 检查发现宿主机缺少 VMware Network Adapter VMnet8 网卡。

解决：在 VMware 的虚拟网络编辑器中点击“还原默认设置”，并在 Windows 网络适配器中启用该网卡，成功恢复通信。

2.MongoDB 官方源下载极慢导致安装失败

排查：apt-get 安装卡在 0% 长时间不动，更换国内源也经常报 GPG 公钥错误。

解决：放弃包管理器，直接在浏览器下载官方的 .tgz 二进制压缩包，解压后手动配置 bin 环境变量，实现了极速安装。

3.副本集初始化时一直报错“不是主节点”

排查：节点启动后，由于选举机制的超时或权重问题，导致三个节点一直处于 SECONDARY 状态。

解决：利用 Python 脚本连接，强制下发 replSetReconfig 指令提升指定节点的权重并强制触发选举，成功让系统进入稳定运行状态。

7.2 项目反思

虽然本次由于设备性能限制，采用了单机多端口的方式来模拟集群，但在配置 keyFile、初始化 rs.initiate、添加 shard、选择分片键的过程中，我对 MongoDB 的分布式工作原理有了极为深刻的理解。在未来的真实生产环境中，这些经验可以直接无缝迁移到多台物理机的部署上。同时，本次项目也让我体会到了文档型数据库在处理多层级嵌套结构时，相比于传统关系型数据库的绝对优势。

八、 图形化验收与交付

按照任务书要求，使用图形化客户端工具（MongoDB Compass）进行独立验

收。

连接与数据量核对：使用 Compass 连接至 mongodb://192.168.148.11:27017，左侧导航栏选中 library 数据库下的 books 集合。界面显示文档总数为 50 条，与源数据文件完全一致。

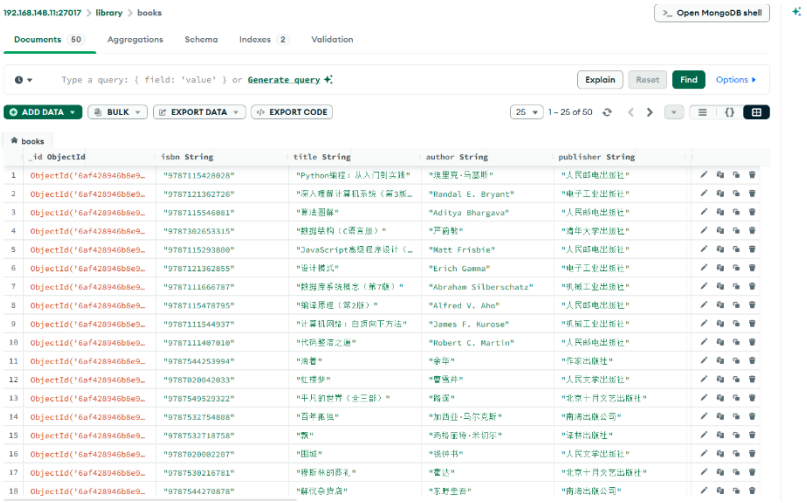


图 6 图形化界面验证

嵌套结构验证：双击 books 集合中的单条文档，展开 location 字段，清晰展示了 floor、shelf、copies 的嵌套子属性，验证了数据模型设计的成功。

192.168.148.11:27017 > library > books

Documents 50 Aggregations Schema Indexes 2 Validation

Type a query: { field: 'value' } or Generate query

ADD DATA BULK EXPORT DATA EXPORT CODE

books location []

	_id ObjectId	floor String	shelf String	copies Int32
1	ObjectId('6af428946b8e9…	"二楼专业图书区"	"A-07"	2
2	ObjectId('6af428946b8e9…	"二楼专业图书区"	"A-03"	2
3	ObjectId('6af428946b8e9…	"二楼专业图书区"	"A-17"	3
4	ObjectId('6af428946b8e9…	"二楼专业图书区"	"A-04"	1
5	ObjectId('6af428946b8e9…	"二楼专业图书区"	"A-02"	3
6	ObjectId('6af428946b8e9…	"二楼专业图书区"	"A-09"	2
7	ObjectId('6af428946b8e9…	"二楼专业图书区"	"A-19"	1
8	ObjectId('6af428946b8e9…	"二楼专业图书区"	"A-18"	3
9	ObjectId('6af428946b8e9…	"二楼专业图书区"	"A-02"	1
10	ObjectId('6af428946b8e9…	"二楼专业图书区"	"A-05"	3
11	ObjectId('6af428946b8e9…	"一楼中文图书区"	"B-04"	1
12	ObjectId('6af428946b8e9…	"一楼中文图书区"	"B-19"	1
13	ObjectId('6af428946b8e9…	"一楼中文图书区"	"B-20"	1
14	ObjectId('6af428946b8e9…	"一楼中文图书区"	"B-18"	3
15	ObjectId('6af428946b8e9…	"一楼中文图书区"	"B-03"	3
16	ObjectId('6af428946b8e9…	"一楼中文图书区"	"B-04"	3
17	ObjectId('6af428946b8e9…	"一楼中文图书区"	"B-16"	1
18	ObjectId('6af428946b8e9…	"一楼中文图书区"	"B-03"	3

图 7 location 字段

导出数据交付：执行 Python 脚本，将 category_stats 统计结果成功导出为 category_stats.json 文件，随项目代码一并提交。



图 8 json 导出